Fine-tuning MACE with the mdstats campaign CLI

The campaign CLI turns source-certified VASP data into a current selected training set and a fresh production model. It keeps scientific authorities, restart state, replay identity, checkpoint evidence, and currentness checks in one disk-backed campaign.

Run commands from the repository root:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml <command>
```

Current workflow

The public scientific lifecycle is exactly:

```
init -> doctor -> prepare -> select-target-size -> cross-validate -> train-production
```

Post-production qualification of the finished product is a separate downstream family:

```
qualification status | qualification run | qualification activate-locked
```

`storage` is an orthogonal artifact-management command whose eligibility comes from the real owners, not from pathnames. `status` reports the training lifecycle, and `advance` chooses the next safe owner from that lifecycle; neither introduces another scientific state machine, and `advance` never runs qualification or opens locked evidence.

1. Create a configuration

```
python tools/mdstats-mlff-campaign.py --config campaign.toml init
```

Set the paths and foundation identity in the generated file. A minimal campaign provides a training root, an inspected foundation checkpoint, and one selected replay corpus:

```
[paths]
training_root = "/path/to/LTA_training"
foundation_model = "/path/to/mace-foundation.model"
replay_set = "/path/to/replay.extxyz"
```

The generator exposes the current target-size policy explicitly:

```
[target_data.size_convergence]
target_size_power_min = 7
target_size_power_max = 14
evaluation_size_powers = [8, 9, 10]
fidelity_epochs = [1, 3, 10]
```

The power ceiling is configuration, not a hidden fixed-size scientific limit. The available population still bounds materializable candidates. Optimizer seeds are authored only by the sole enabled training method.

Current post-selection CV settings are authored under one canonical section:

```
[post_selection.cv]
fold_count = 2
partition_seed = 7
seeds = [11]
max_num_epochs = 2
acceptance_maximum = 0.5

[post_selection.production]
seeds = [5]
```

Pre-target fold controls are not generated as target-size authority. Historical read-only fields may be accepted for a compatible existing campaign, but they cannot silently become current scientific settings.

The learned model supports `single` (FP32) or `double` (FP64) precision. mdstats scientific reductions, reference fitting, geometry, and persistent bookkeeping remain FP64 in either mode. The selected acceleration backend and MACE runtime identity are bound by `doctor` and the campaign protocol.

2. Check inputs and runtime

```
python tools/mdstats-mlff-campaign.py --config campaign.toml doctor
```

Do not continue until blocking source, manifest, foundation, replay, backend, and runtime checks pass. `doctor` records the manifest approval and the exact runtime realization used by later owners. A missing accelerator or unavailable long-production environment is reported as unavailable; it is not converted into a qualification pass.

3. Prepare the current substrate

Review the source manifest first:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare
```

When the manifest needs operator approval, approve that exact digest:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare --approve-manifest
python tools/mdstats-mlff-campaign.py --config campaign.toml prepare
```

Use `--refresh-inferences` before approval when source metadata or inference inputs changed. Preparation is restartable and owns only the current neutral substrate:

```
source/frame/label authority
  -> protected statistical relations
  -> one P_train/M3 split and pi_train/pi_eval
  -> one common target-size preparation
```

`prepare` does not select a target size, train a candidate, rank a checkpoint, or publish production. The configured ladder is an experiment definition, not a decision. The one-time destructive cutover rejects obsolete derived target-size records before reuse and quarantines them rather than migrating them. Those records are never translated into current authority.

Preparation reuses only lower-level inputs that current owners revalidate. A source or scientific-identity change invalidates the affected generation; provenance-only presentation changes do not change arithmetic. Interrupted work is resumed by rerunning the same command after inspecting `status`.

4. Select the target size

```
python tools/mdstats-mlff-campaign.py --config campaign.toml select-target-size
```

This is the only current screening entrypoint. It is the sole command that trains target-size candidates or decides `N_selected`. Every candidate is an exact prefix of one deterministic `pi_train` order, so the selected set is

```
T_selected = pi_train[:N_selected]
```

The screen uses the configured power range, direct nested evaluation populations `M1 subset M2 subset M3`, the configured `fidelity_epochs`, and the ordered seeds from the sole enabled training method. It runs the authenticated continuation:

```
n1 / M1 -> n2 / M2 -> n3 / M3
```

The current default is `(n1, n2, n3) = (1, 3, 10)`. Boundaries are continuation points; an earlier better checkpoint cannot replace the prescribed endpoint. The reducer first narrows the qualified population, then freezes one size and its exact membership or records a typed scientific failure.

Replay metrics, post-selection CV, physical-observable evidence, and downstream qualification cannot rank or tie-break a size. A terminal nonconvergence at the configured ceiling is a scientific result, not permission to invent a rescue size. An incomplete but nonterminal run remains resumable.

The selected size and membership are not editable fields. Every current read re-derives them from authenticated reducer state and `pi_train`; divergence fails closed.

5. Validate the frozen method

```
python tools/mdstats-mlff-campaign.py --config campaign.toml cross-validate
```

Cross-validation starts only after selection and consumes exactly `T_selected`. It validates the training method, not the amount of data. The configured `K >= 2` folds preserve the P1 split-exclusion and correlation relations; every required fold and optimizer seed must pass the target-only acceptance predicate.

Fold partitions are constructed inside the already frozen selected set. A fold may fit training-only transforms from its own training partition, freezes its representative on its authorized monitor, and evaluates the held-out partition only afterwards. Held-out fold results cannot change `N_selected`, membership, checkpoint policy, or the method definition. Replay remains a separate admissibility/retention concern and supplies no ranking credit.

A missing or failing fold is a methodological failure. It leaves selection evidence unchanged and does not authorize final production; it is not replaced by a mean, majority, best-seed, or partial-fold result.

6. Train fresh final production

```
python tools/mdstats-mlff-campaign.py --config campaign.toml train-production
```

Final production starts from the accepted foundation with fresh optimizer, RNG, and run state. It trains the complete exact `T_selected` using the method accepted by cross-validation and `[training].max_num_epochs`. Screening and CV checkpoints are not production parents, even when their numeric seed or size matches.

The production horizon is independent of the screen's `n3`. A production-only configuration change invalidates production descendants while leaving the selected binding and accepted CV evidence current. Both post-selection owners re-authenticate currentness before work and publish only under a commit-time currentness fence.

`train-production` finishes by publishing the **final-production publication decision**: which of the completed seeds constitute the released product, under the configured `[post_selection.production].committee_policy`. Both policies are supported. `all_qualified_final_seeds` publishes every required seed whose already-frozen representative checkpoint is admissible; `single_best_final_seed` ranks those same already-frozen representatives with the accepted target-only EVAL2 ordering over the frozen M3 development evidence and publishes one. The decision is taken here, before any qualification evidence exists, and nothing downstream can change it.

The training lifecycle ends at that publication. Everything after it validates the finished product without being able to change it.

7. Qualify the frozen product

Qualification consumes the final-production publication that `train-production` already froze. It never creates, reorders, or shrinks that publication, and it owns no target-size, cross-validation, production, checkpoint, seed, or member decision. Every threshold under `[qualification]` is fixed in the configuration before any product outcome is observed.

```
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification status
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification run
```

`qualification run` executes or resumes the nonlocked components for the exact frozen publication:

- **deployment parity** - the published model is exported at the canonical target head, converted to the deployed ML-IAP artifact at that same head, executed through the real supported LAMMPS runtime, and compared against the authenticated in-framework model under dtype-justified tolerances, including stress whenever the product actually has a stress channel. Stress applicability is decided from the accepted training objective, the reference labels, the model, periodicity, and the runtime - not from a configuration switch, so `stress_required` can insist on qualifying an available channel but nothing can quietly suppress one;
- **local PES** - deterministic symmetric displacement modes on a candidate-independent `OUTER_MONITOR` base cohort, checked for pointwise force agreement, restoring sign, and stiffness/curvature against matched external references;
- **relaxation** - fixed-cell minimization compared against matched reference relaxations, with protected-topology safety judged separately from geometric fidelity;
- **dynamics** - bounded NVT warm-up and NVE propagation through the deployed artifact, started from the authenticated reference-relaxed geometry of each physical base, and checked for NVT and NVE temperature behaviour, energy drift, minimum pair distance, force bounds, and protected topology, displacement, bond, and angle degradation. Topology damage must persist for a configured number of consecutive samples before it rejects, so one noisy sample is not mistaken for a broken framework;
- **calibration** - uncertainty calibration of the exact frozen committee on the reserved `UNCERTAINTY_CALIBRATION` role, or an explicit `not_applicable` for a single-model product with no accepted uncertainty estimator.

Three outcomes are not failures and must not be read as one:

- `waiting_for_reference` means the external first-principles evidence the frozen physical plan asked for has not been supplied. The exact request is written to the reference root as `reference-request.json`; supply a matching `reference-bundle.json` and rerun. Nothing is ever passed on absent evidence.
- `not_applicable` means the frozen policy declares a component inapplicable to this product.
- an *unavailable* supported deployment runtime blocks the deployment claim rather than passing or rejecting it. Executing *an* ML-IAP model and executing *this MACE product* are separate capabilities, and only the second one proves the product path; `qualification status` reports both.

`[qualification.reference].protocol` must name a real reference method before any reference-dependent component runs. The generated placeholder fails closed rather than letting an unlabelled bundle become a release claim.

A component rejection rejects that exact published product. It never selects a different seed, checkpoint, or committee member, never shrinks a committee, and never reaches back into target-size selection, cross-validation acceptance, or production training.

The one-shot locked test

```
python tools/mdstats-mlff-campaign.py --config campaign.toml qualification activate-locked --confirm
```

This is the only path that opens the reserved `LOCKED_INTERPOLATION_TEST` cohort. It requires `--confirm`, requires every mandatory nonlocked component to have already passed, and is refused a second time for the same publication and cohort. Activation is irreversible: once the cohort is revealed it is never a fresh locked test again, whatever the policy is later changed to, and a retrained product needs genuinely new independent evidence.

A locked pass produces the terminal `release_qualified` verdict; a locked failure rejects the exact published product.

Activation is an irreversible *open* event, not a claim that the evaluation finished. If the process dies between opening the cohort and publishing the result, rerunning `activate-locked --confirm` resumes the same activation and finishes the same test; it never opens a second one. Only a genuinely completed terminal result makes a further activation a rejected duplicate.

Supplying a new reference bundle for the same frozen request re-runs local PES, relaxation, and dynamics - the components that consume it - and reuses the deployment and calibration evidence, which do not.

Where the evidence lives

```
campaign/.mdstats/qualification/g<N>/objects/    immutable release evidence
campaign/.mdstats/qualification/g<N>/attempts/   attempt state and scratch
campaign/qualification-references/<plan>/        reference request and bundle
```

Durable qualification evidence is release evidence, not reconstructible scratch: storage cleanup never reclaims it, and it also cannot reclaim an artifact an in-flight qualification attempt still references.

Each attempt also records what it actually cost - elapsed time overall and per component, workspace free space and the attempt's own footprint at start and end, peak memory, and accelerator telemetry where available - and the terminal record points at it. Before each component materializes artifacts or scratch, the campaign's existing `[execution].minimum_free_disk_gib` reserve is checked; an attempt that cannot proceed safely aborts rather than changing anything scientific. Mixed periodic boundaries are executed exactly as configured, so a `[True, True, False]` system runs as itself rather than being silently coerced.

Inspect and resume

```
python tools/mdstats-mlff-campaign.py --config campaign.toml status
python tools/mdstats-mlff-campaign.py --config campaign.toml advance
```

The workspace stores the durable campaign state and content-addressed payloads:

```
campaign/
|-- campaign-manifest.json
|-- .mdstats/campaign.sqlite3
|-- .mdstats/                    # current-generation records and caches
|-- runs/                         # authorized checkpoints and logs
|-- models/                       # current production model evidence
`-- results/                      # bounded summaries and cleanup reports
```

Rerunning a current owner is safe: complete authenticated cells are reused, stale or corrupt derived caches are rebuilt, and a superseded owner cannot publish a current descendant. Close stores/processes cleanly before inspecting or rerunning another owner.

Storage management

Storage is orthogonal to the scientific lifecycle. What may be touched is decided by the real P1-P7 owners - never by a pathname, a report label, a stage name, a file age, or a process id. Start with a read-only report:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage report
python tools/mdstats-mlff-campaign.py --config campaign.toml storage report --deep
```

`storage report` reads the owner views and bounded physical metadata; it tells you which owner holds each family, what is current, and how much each action could reclaim. It is bounded - one `lstat` per owner artifact, no subtree walk - so a directory's total size is reported as unknown rather than guessed, and `--deep` is the explicit opt-in to an exact recursive physical audit.

Both are read-only, and read-only here means the command changes nothing at all: it does not create the workspace or the campaign database, it writes no report file into `results/`, and it does not even warm the SHA-256

receipt cache. The report is printed, not deposited. Running it against a directory that was never prepared tells you the campaign is uninitialized instead of initializing it.

Authority to change anything comes only from the invocation you are running. `--apply` on this command line authorizes the mutation and the subcommand you typed selects the action; a persisted `apply` or `action` key under `[storage]` in your TOML is rejected outright rather than obeyed. Every consequential action plans first and mutates only when you authorize it:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier safe --dry-
run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier safe --
apply
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier cache --
dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage cleanup --tier cache --
apply
```

`--tier safe` loses no scientific, restart, qualification, locked, or acceleration-cache capability: it removes only artifacts an owner has positively released, such as external record payloads no campaign state row references. `--tier cache` adds eviction that an owner can certify as exactly reconstructible right now *and* can certify nothing is currently depending on. Reconstructibility alone is not enough. The normalized frame cache is reported as exactly reconstructible - rebuilding costs one source read per DATA2 run and reproduces the identical authenticated cache - but P1 exposes no liveness seam that could prove no reader is using it at this instant, so it is retained by both tiers and reported as `cache_reconstructible = true, cache_evictable = false`. The SHA-256 receipt store is likewise accounted as a cache and retained. Anything no owner can certify is retained, so a `cache` run today is legitimately a no-op on most campaigns.

Cleanup also plans campaign-state maintenance as its own action rather than as a side effect: bounding diagnostic events and rewriting the SQLite file happen only when SQLite's own free-space accounting says the rewrite is worth its cost, and only when there is room for the temporary copy it makes.

Cold archive turns owner-declared *historical* bulk into a reversible, authenticated, identity-keyed cold representation:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive create --dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive create --apply
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive list
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive verify <identity>
python tools/mdstats-mlff-campaign.py --config campaign.toml storage archive restore <identity>
--apply
```

Hot bytes are removed only after the archive is authenticated and cataloged, and only for artifacts no current or restartable owner needs hot - a checkpoint the current qualification publication still authenticates is never archived out from under it. Only descendants the owning component certifies as its own are collected: a file something else dropped inside an otherwise eligible run tree is left alone, and its presence withholds authority over that whole tree until you remove it.

`--root` may narrow the selection to one eligible artifact. It may not widen it: naming a parent directory is rejected rather than reinterpreted, because a parent sweeps in siblings no owner released.

Restoring an archive brings the bytes back as *historical* evidence; it never promotes anything to current, and it never changes the permissions or ownership of a directory that already existed. If a reclamation was interrupted, `storage archive reclaim <identity> --apply` resumes it against a freshly authenticated archive.

Deduplication is a representation change only:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml storage deduplicate --dry-run
python tools/mdstats-mlff-campaign.py --config campaign.toml storage deduplicate --apply
```

It shares one inode directly between byte-identical files whose owner certifies both immutability and matching filesystem metadata. There is no separate content store, so deleting the last remaining name releases the bytes with nothing left to collect. Equal bytes alone are never enough; a file already hardlinked to something outside the campaign is never chosen as the shared inode, and a cross-device or unsupported filesystem simply keeps the duplicates.

Every action protects external inputs, current scientific records, selected checkpoints, restart evidence, and diagnostics. The retired `recompute` and `compact` loss tiers are not current product authority and are rejected by name.

Interpreting outcomes and limitations

The durable result is the authenticated chain of source identity, neutral substrate, target-size experiment, selected binding, CV acceptance, and final production identity. A target-size scientific failure is terminal evidence and does not expose a production next action. A missing accelerator, unavailable target-machine run, or absent downstream qualification is reported as deferred or unavailable rather than silently passed.

The P6 implementation provides current functional, restart, and public-surface closure. It does not establish GPU, long real-data, M-ladder decision- preservation, or downstream release qualification.